BÀI TẬP NHÓM GIỮA KỲ
8-PUZZLE PROBLEM

Bài toán: Hoán vị các ô 0,1,...,8 trên bàn cờ 3x3 để đưa về GOAL
Câu hỏi trung tâm: Làm thế nào để giải bài toán 8-Puzzle tối ưu chi phí và kiểm chứng tính đúng đắn của thuật toán A* cùng các hàm Heuristic?

Giảng viên: TS. Nguyễn Bích Vân
Nhóm thực hiện: Nhóm 1
Lớp: AI2014_1

HAI MÔ HÌNH COST, TRẠNG THÁI VÀ GOAL

Trạng thái s: Tuple hoán vị của 0, 1, . . . , 8 (Số 0 là ô trống).
U, D, L, R là hướng của ô trống.

Mô hình	Chi phí một bước	Mục tiêu
unit	U/D/L/R đều bằng 1	Ít bước nhất = ít chi phí nhất
directional	U/D bằng 1; L/R bằng 2	Ít tổng chi phí nhất, không chỉ ít bước nhất

Trạng thái GOAL: Không mặc định đích luôn là G khi đã thay dữ liệu

Khi nào bài toán có lời giải ?
Đếm cặp nghịch thế so với thứ tự các quân khác 0 trong đích. Bài toán có lời giải khi và chỉ khi số nghịch thế chẵn.

1	2	3
4	5	6
7	8	

Đích chuẩn G

1	2	3
4		6
7	5	8

Ví dụ làm quen D

THUẬT TOÁN A*

- Chọn nút theo f(s) = g(s) + h(s); hòa xử lý theo FIFO
- Lưu best_g bằng heap, loại bỏ bản ghi cũ khi pop
- Chỉ dừng và dựng đường khi nút lấy ra trùng GOAL (không kiểm tra lúc sinh con)

$$h_1(s) = \#\{i \neq 0 : \text{ quân } i \text{ sai vị trí}\},$$

$$h_2(s) = \sum_{i=1}^{8}(|r_i - r_i^*| + |c_i - c_i^*|),$$

$$h_3(s) = \sum_{i=1}^{8}(|r_i - r_i^*| + 2|c_i - c_i^*|).$$

3 hàm Heuristic

Bất đẳng thức chấp nhận được: $h_1 \leq h_2 \leq h^*(unit)$

$h_1 \leq h_2 \leq h_3 \leq h^*(directional)$

ĐỐI CHIẾU TÍNH TAY VỚI TRACE

H = (1, 3, 6, 5, 0, 2, 4, 7, 8)

Lần	Nhãn / trạng thái lấy ra	g	h	f	Khớp tay–code?
1	S = (1,3,6,5,0,2,4,7,8)	0	8	8	Khớp
2	L = (1,3,6,0,5,2,4,7,8)	1	7	8	Khớp
3	R = (1,3,6,5,2,0,4,7,8)	1	7	8	Khớp
4	LD = (1,3,6,4,5,2,0,7,8)	2	6	8	Khớp
5	RU = (1,3,0,5,2,6,4,7,8)	2	6	8	Khớp
6	LDR = (1,3,6,4,5,2,7,0,8)	3	5	8	Khớp

MỘT SO SÁNH HEURISTIC

Tại M2:

- Start = (1, 3, 8, 7, 2, 4, 5, 6, 0)
- Goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)
- Cost mode: directional - Order: UDLR.

.

Case	N A*/h2	N A*/h3
M2	91	33

MỘT PHẢN VÍ DỤ BFS

Cost mode: directional

Case	BFS: bước / cost	UCS: bước / cost
M3	20/30	20/28

BFS chỉ xét theo số lượng bước đi ngắn nhất chứ không xét đến trọng số cạnh.

CÁC TEST TỰ VIẾT

STT	Tên test
1	test_trang _ thai _ dau_ trung _dich
2	test_chuyen_ dich_ mot_ buoc_ngang _doc
3	test_dich_tuy_chinh
4	test_heuristic_bang_0_tai_ dich_va_bo_qua_o_trong
5	test_vo_nghiem_do_parity
6	test_du_lieu_sai_va_hanh_dong_vuot_bien
7	test_duong_di_hop_le_va_dau_vao_bat_bien
8	test_bfs_khong_toi_uu_gia_o_directional
9	test_doi_chieu_astar_zero_va_ucs
10	test_doc_lap_trace_va_gioi_han_tai_nguyen

2 LỖI HEURISTIC

Đếm ô trống

Phá vỡ tính chấp nhận được vì tồn tại trạng thái mà h > h*.

Goal hard-code

Heuristic dùng vị trí đích cố định theo GOAL chuẩn, bỏ qua goal tùy chỉnh, không hợp lệ với goal bất kỳ.

DỮ LIỆU NHÓM

Goal = (1, 2, 3, 4, 5, 6, 0, 7, 8)

Nội dung	N1	N2
Start	(6, 1, 3, 5, 0, 2, 4, 7, 8)	(4, 1, 0, 5, 3, 2, 6, 7, 8)
Seed	2026	2026
Lịch sử	("U","R","U","L","D","R","R","U","L","D","L","U","R","R","D","L")	("R","U","R","U","L","L","D","D","R","U","L","D","R","U","R","U")
Số bước xáo trộn	16	16

ĐÓNG GÓP

Họ và tên	Việc thực hiện	Hai test phụ trách	Kiểm tra chéo
Đoàn Mai Trúc Linh	Thực hành với mã hoàn chỉnh & tạo dữ liệu và đóng gói	Test 1 - 2	BaoCao.pdf và TinhTay.pdf
Trần Thùy Dương	Mô hình hóa bài toán Mã giả A* & Chứng minh Heuristic	Test 3 - 4	du_lieu_nhom.json và tu_tao.csv
Hoàng Thị Ngọc Sơn	Tính tay Vết chạy A* & Phân tích phản ví dụ	Test 5 - 6	test_nhom.py
Phạm Hồ Minh Thư	Phân tích thực nghiệm & Nhận xét báo cáo	Test 7 - 8	ThuyetTrinh.pdf
Nguyễn Thanh Bình	Slide Thuyết trình & Lập trình Kiểm thử	Test 9 - 10	README_NHOM1.md

GIỚI HẠN

- Số lượng bàn kiểm thử còn hạn chế, kết quả chưa đại diện cho toàn bộ không gian trạng thái của bài toán 8-puzzle.
- Thay đổi thứ tự sinh trạng thái con UDLR và cách xử lý trạng thái lặp có thể khiến BFS tìm được một đường đi khác có cùng số bước nhưng chi phí khác.

KẾT LUẬN

Heuristic và mở rộng nút

Ở mô hình unit, A* với h_2 (manhattan) mở rộng ít nút hơn h_1 (misplaced) mà vẫn tối ưu. Ở directional, h_3 (weighted) giảm tiếp so với h_2 .

Số bước và chi phí

Ở directional, BFS tối thiểu số bước nhưng không tối thiểu chi phí do L/R giá 2.

Một lỗi đã sửa hoặc điểm hiểu rõ hơn

Việc kiểm thử độc lập giúp chứng minh được BFS không tối ưu

Nguồn dùng lại:

Bộ mã nguồn cốt lõi (gồm các tệp search.py, puzzle.py, run.py, tao_du_lieu.py,...) và các thuật toán tìm kiếm (bfs, ucs, astar)